couplr: Optimal Matching with Verifiable Certificates and Sparse Edge Generation

by Gilles Colling

Abstract Matching two groups of units so that the total cost of the chosen pairs is as small as possible is the linear assignment problem, which sits underneath causal-inference matching, experimental design, task allocation and image alignment. Two things bound what software built on it can offer. An answer arrives without a proof, since a solver reports the state it terminated in rather than a statement about the matching that a user can check. And the graph of admissible pairs is built before it is solved, which puts the size ceiling at the memory that complete graph occupies. We present the couplr package, which addresses both. Every matching design compiles into one internal flow model, so node potentials come back from all of them and can be checked against the linear-programming optimality conditions. On those potentials the package solves without building the graph: each unit starts with its nearest admissible partners, the omitted pairs are priced against the duals the sparse solve returns, and pairs enter until none prices in, at which point the duals certify the sparse solution optimal for the complete problem. Matching 16,667 treated units to 33,333 controls holds 0.29 percent of the pairs and returns the assignment the dense solve returns, certified. Warm starting the same loop traces a matching frontier over a swept constraint at a fraction of the cost of solving each point from cold. A data-frame interface, nineteen assignment algorithms, balance diagnostics and cardinality matching with an optimality gap sit on the same core. couplr is available on CRAN.

1 Introduction

Many data-analysis tasks come down to the same question: given two sets of objects and a cost attached to every possible pair, which pairing has the smallest total cost? A researcher building an observational study pairs treated units with controls that resemble them on measured covariates. A scheduler assigns staff to shifts, an ecologist pairs surveyed plots across two time points, and an image-processing pipeline maps the pixels of one picture onto the pixels of another. In each case the mathematical object is the linear assignment problem, and the quality of the answer depends on solving it to optimality rather than greedily.

R has good software at both ends of this problem, in two separate groups of packages. On the statistical side, **MatchIt** (Ho et al., 2011) provides a common interface to several matching designs and delegates optimal pair and full matching to **optmatch** (Hansen, 2007). **optmatch** formulates those designs as minimum-cost flow problems (Hansen and Klopfer, 2006) and supports more than one back end, currently RELAX-IV and LEMON, selected through its `solver` argument with a choice of four LEMON algorithms, and stores sparse distances as an `InfinitySparseMatrix` whose non-finite entries are the pairings it will not consider. **Matching** (Sekhon, 2011) provides multivariate and propensity-score matching, with `GenMatch()` using genetic search to optimise covariate weights. **designmatch** (Zubizarreta et al., 2024) formulates balance-constrained designs as integer programs and defaults to the open-source HiGHS solver. **rcbalance** (Pimentel, 2022) matches under refined covariate balance constraints (Pimentel et al., 2015), taking a distance structure that lists the permissible controls of each treated unit and solving the resulting network as a minimum-cost flow problem. **quickmatch** (Sävje et al., 2024) aims at large data sets with generalized full matching, which its own description calls near-optimal. **cobalt** (Greifer, 2025) supplies balance diagnostics across packages. What these expose is the matching design; the assignment problem underneath is reached through a solver argument at most, and alternative optima and a bottleneck objective sit outside all of them. Statements about other packages here and in Table 5 were assessed on 2026-08-09 against **MatchIt** 4.7.2, **optmatch** 0.10.8, **Matching** 4.10-15 and **designmatch** 0.5.5, and against the reference manuals of **rcbalance** 1.8.8 and **quickmatch** 0.2.3 on 2026-08-26.

On the algorithmic side, **clue** (Hornik, 2024) exposes the Hungarian method through `solve_LSAP()`, and **lpSolve** (Berkelaar et al., 2024) solves the assignment problem as a general linear program. These return an optimum for a cost matrix the user has already built; covariate handling, calipers and balance tables sit outside their scope.

A workflow that needs both halves therefore threads several packages by hand. Matching on observed covariates instead of a propensity score, imposing calipers on more than one variable at a time, blocking, and choosing the solver deliberately are all available in R, each in a different package.

Two further limits are common to both groups, and they are the ones this article is about. The first concerns what a result says about itself. A solve returns a matching and a status, and the status records the state the solver terminated in. Whether that matching is the optimal one is a separate question with a checkable answer, since linear programming supplies optimality conditions that a third party can test on the returned solution, and none of the packages above exports a function that returns the dual variables those conditions need or that checks the conditions. **optmatch** comes closest: a `fullmatch()` result carries the node prices its flow solver ended on in an undocumented attribute, and the function that would evaluate the primal against them is internal. What it documents in their place is a solver tolerance, `tol` times the number of subjects bounding how far the returned match may differ from an optimal solution of the specified problem.

The second concerns size. Sparse matching is a developed literature, and what its methods have in common is that the sparse network is settled before the matching is solved over it. **rcbalance** takes a per-unit list of permissible controls built from exact groups and a caliper. Yu et al. (2020) choose a propensity-score caliper by an iterative form of Glover's algorithm, radically reducing the candidate matches, and then match optimally in the much sparser graph that leaves. Yu and Rosenbaum (2022) grade the potential pairings and admit lower grades only as far as they are needed, so that only sparse networks are built, stored and optimised. In each case the rule deciding which pairs exist is fixed in advance of the solve that uses them, and a network too sparse drops pairings it should have considered. What is missing is a rule read off the solution in hand, and a statement about the pairs it never built.

We present the **couplr** package (Colling, 2026), which treats the assignment problem itself as the organising object and builds the workflow around it. A user who starts from a data frame with covariates and a treatment indicator, and a user who starts from a cost matrix, enter through different functions and reach the same solver core. We contribute five things to R:

1. A common compilation layer for statistical matching designs. One-to-one, k-to-one, with replacement, blocked and full matching are descriptions of the same internal flow model, differing in the capacities their arcs receive, so what is written once serves all of them.
2. Certificates rather than reported status. `verify_assignment()` and `verify_flow()` take a solution and check primal feasibility, dual feasibility, complementary slackness and objective equality, returning each condition separately. The check is independent of the solver that produced the solution, and a matching from a solver that returns no duals of its own can be certified against duals obtained separately. `verify_assignment()` decides those conditions in exact arithmetic where the potentials allow it, and names the arithmetic its verdict was reached in.
3. Dual-certified adaptive edge generation. Under `memory_mode = "implicit"` the complete graph is never built: a sparse restricted problem is solved, the omitted pairs are priced against its duals, violating pairs are added, and the loop repeats until none prices in, at which point the duals certify optimality for the complete problem.
4. Warm-started matching frontiers. `match_path()` sweeps one constraint over a sequence of values, resuming each point from the matching the point before it found, and returns a certificate per point.
5. Nineteen assignment algorithms implemented from scratch in C++ and reachable by name, covering augmenting-path, auction, cost-scaling and network-flow families, together with k-best assignments, bottleneck assignment, entropy-regularised transport, and cardinality matching that reports an optimality gap against the largest sample its constraints admit.

We state the problem couplr solves, the conditions that certify a solution to it, and the algorithm families that reach one. A matching workflow and the designs around it follow, then the three pieces that carry the contributions above: the flow model every design compiles into, the edge-generation loop built on its duals, and the frontier that warm-starts that loop. The software design follows in detail: object model, C++ organisation, dispatch rules, memory modes, verification and dependency footprint. A performance section reports per-solver timings, a balance comparison against **MatchIt** and **optmatch** on the LaLonde data, a scaling comparison, and what the edge-generation loop builds and pays. Results were produced with couplr version 1.6.2.

2 The assignment problem behind matching

Let $C \in \mathbb{R}^{n\times m}$ be a cost matrix whose entry C_{ij} is the cost of pairing row $i \in \{1,\ldots,n\}$ with column $j \in \{1,\ldots,m\}$, and take $n \le m$; couplr transposes the problem internally when it is not and maps the assignment back afterwards, so the user never sees the transpose. Write $X \in \{0,1\}^{n\times m}$ for the assignment matrix, with $X_{ij} = 1$ when row i is matched to column j. The linear assignment problem is

$$\min_X \sum_{i=1}^{n}\sum_{j=1}^{m} C_{ij}X_{ij} \quad \text{subject to} \quad \sum_j X_{ij} = 1\ \forall i, \qquad \sum_i X_{ij} \le 1\ \forall j. \tag{1}$$

Every row is matched and every column is used at most once. The row constraint is an equality: relaxing it to $\sum_j X_{ij} \leq 1$ would leave $X = 0$ optimal for any non-negative cost matrix, which is not the problem a matching application poses.

In a matching application, rows are treated units, columns are controls, and C_{ij} is a distance between covariate vectors. In a scheduling application, rows are workers, columns are shifts, and C_{ij} is a cost or the negative of a preference score.

Forbidden pairs are encoded as $C_{ij} = \infty$, and enough of them make (1) infeasible, since no assignment can then reach every row. couplr solves a lexicographic relaxation in that case: the largest number of matched rows first, then the smallest total cost among the assignments attaining that number. The section on constraints below describes how that problem is reached and what it costs.

The constraint matrix of (1) is totally unimodular, so the linear programming relaxation has integral optima (Birkhoff, 1946; Burkard and Çela, 1999) and the combinatorial problem can be solved by primal-dual methods rather than by branch and bound. The dual assigns a potential u_i to each row and v_j to each column, and is

$$\max_{u,v} \sum_{i=1}^{n} u_i + \sum_{j=1}^{m} v_j \quad \text{subject to} \quad u_i + v_j \leq C_{ij} \; \forall (i,j), \qquad v_j \leq 0 \; \forall j. \tag{2}$$

The sign condition is what the column inequality in (1) costs: the dual objective sums v over every column while an assignment pays only for the ones it uses. When $n = m$ none can be left out, the column constraints bind, and v is unrestricted in sign; `verify_assignment()` imposes $v_j \leq 0$ only when $n < m$, since Jonker-Volgenant returns free-sign potentials on a square problem and they certify optimality there.

Complementary slackness ties a pair of potentials to a particular X:

$$X_{ij} = 1 \implies u_i + v_j = C_{ij}, \qquad \textstyle\sum_i X_{ij} = 0 \implies v_j = 0. \tag{3}$$

The reduced cost $C_{ij} - u_i - v_j$ is a lower bound on what forcing pair (i,j) would add to the total rather than the whole of it, because forcing one pair can displace others.

Certifying a solution

Dual feasibility on its own says nothing about a particular X. What certifies optimality is X primal feasible, (u,v) feasible for (2), and both halves of (3) holding. In exact arithmetic those three settle the primal and dual objectives as well, so objective equality is a reported check rather than a condition of its own. `verify_assignment()` reports four checks and gives each separately, so a failure names the check it failed on.

```
set.seed(2)
cost <- matrix(runif(120), nrow = 6, ncol = 20)
verify_assignment(assignment(cost), cost)

#> Assignment certificate
#> ======================
#>
#>   primal_feasible          TRUE
#>   dual_feasible            TRUE
#>   complementary_slackness  TRUE
#>     matched arcs tight     TRUE    (max slack 0.000e+00)
#>     unmatched columns free TRUE    (max |v_j| 0.000e+00)
#>   duality_gap              0.000000e+00
#>   certified_optimal        TRUE
#>   arithmetic               exact, no tolerance
#>
#>   primal objective  0.2821665586
#>   dual objective    0.2821665586
#>   matched           6 of 6 rows, 20 columns
```

Two properties make this a check rather than a report. It is written against the solution, not against the solver, so it takes an assignment, a cost matrix and a pair of potentials and never consults the code that produced them. And optimal duals are shared by all optimal solutions of a linear program, so a matching from one solver can be certified against potentials obtained from another. That is what puts the solvers returning no duals of their own inside the scope of the check.

Complementary slackness is where a plausible verifier goes wrong, and the rectangular case is where it shows. A verifier that asserts the first half of (3) alone, tightness on matched pairs, accepts solutions that are not optimal. In our own prototyping a perfect, dual-feasible, matched-tight solution passed such a check while its primal cost stood 0.4% above the optimum, because one unmatched column retained $v_j < 0$ and so contributed a spurious term to the dual objective. `verify_assignment()` asserts both halves, and reports them separately as `cs_matched_tight` and `cs_unmatched_free`.

The arithmetic the conditions are decided in is the other place where a certificate can overstate what it knows. Each of them is the sign of $C_{ij} - u_i - v_j$, and every quantity in that expression is a double, which is a rational number, so each sign has an exact answer. Deciding them within a band around zero instead makes the verdict a statement about a neighbourhood of the instance rather than about the instance: a certificate at tolerance ϵ accepts potentials violating dual feasibility by ϵ on any pair and missing tightness by ϵ on any matched arc, which bounds the matching's excess over the optimum by about $2(n+m)\epsilon$ rather than pinning it at zero. At the largest shape in the benchmark below and the default $\epsilon = 10^{-9}$, that is 10^{-4} in the cost units.

Setting the band to zero is not the fix, because the sign of the double evaluation can itself be wrong. With $C_{ij} = 2^{-60}$, $u_i = 1$ and $v_j = -1$ the first subtraction rounds to -1 and the expression returns zero on a pair whose reduced cost is positive; at $C_{ij} = -2^{-60}$ it returns zero on a pair whose reduced cost is negative, which is a dual infeasibility a zero-band check would accept. `verify_assignment()` evaluates the sign exactly instead. It splits each rounded addition into the sum and the part rounding lost, which gives an expansion of doubles equal to the difference in the reals, and takes the sign of that expansion (Shewchuk, 1997). A rounding-error bound decides which pairs are far enough from zero for the double evaluation to settle them, so the expansion runs on the pairs near tightness rather than on all nm.

Removing the band from the two conditions removes it from the conclusion. Objective equality is then not a fourth measurement but a consequence: with matched arcs tight and unmatched columns free, the primal cost is the sum of u over matched rows plus the sum of v over used columns, which is the dual objective exactly when every row is matched. The software reports the gap it computes as an independent numerical cross-check on that reasoning.

An exact certificate states that the matching attains the minimum total cost of the cost matrix as supplied. It is available when the potentials are exactly optimal for that matrix, which is a property of the arithmetic that produced them. Over ten instances per cell, integer costs and uniform costs on the unit interval gave an exact certificate in every instance at 50, 200 and 800 rows square and at 200 rows against 600 columns, and in every instance for each of the eleven solvers we ran at 200 by 200. Euclidean distances between normal clouds gave one in none of them: those potentials miss exact tightness by a relative 2×10^{-16} to 5×10^{-15}, a few units in the last place, which is what rounding in the arithmetic that produced them costs. There the certificate decides the same conditions within a stated tolerance and says that is what it did. Every instance in the grid was certified in one arithmetic or the other. A verdict therefore names the arithmetic it was reached in, and `arithmetic = "exact"` refuses the fallback for a caller who wants the stronger statement or nothing.

Why more than one algorithm

Every solver of (1) returns the same optimal value, so the choice among them is a performance question rather than a statistical one, and the performance depends on the shape of the problem. We group the solvers into four families and a set of special-purpose routines.

Augmenting-path methods grow a matching one row at a time, each time finding a shortest augmenting path in the residual graph under reduced costs. The Hungarian method (Kuhn, 1955; Munkres, 1957) is the classical member; the Jonker-Volgenant algorithm (Jonker and Volgenant, 1987) adds column reduction and auction-style initialisation, which is what makes it fast in practice on dense problems despite an $O(n^3)$ worst case. `lapmod` is the sparse variant, working from an adjacency list so that forbidden pairs cost nothing to carry.

Auction methods (Bertsekas, 1988) let rows bid for columns, raising prices until no row wants to switch. They converge to optimality once the bid increment ε falls below $1/n$ for integer costs, and ε-scaling gives the useful practical variants. Auction is naturally parallel and tolerant of near-degenerate cost structures where augmenting-path methods spend time on long paths.

Cost-scaling methods solve a sequence of increasingly precise problems, each time halving the scale on which costs are rounded and reusing the previous solution (Goldberg and Kennedy, 1995). The Gabow-Tarjan bit-scaling algorithm (Gabow and Tarjan, 1989) belongs here. On a graph with $|V|$ nodes and $|E|$ edges it reaches $O(\sqrt{|V|}\,|E| \log(|V| C_{\max}))$ for integer costs, where $C_{\max} = \max_{i,j} |C_{ij}|$. A complete bipartite graph on n and m units has $|V| = n + m$ and $|E| = nm$, so the bound there is $O(\sqrt{n+m}\;nm \log((n+m)C_{\max}))$. To our knowledge this algorithm did not previously have a publicly available open-source implementation in any language.

Bit-scaling runs on integers, so the conversion is part of the method and its rule is stated rather than left to the caller. Finite costs are shifted so the smallest is zero, multiplied by a scale factor s, and rounded to the nearest integer, with s set so that $Ks(C_{\max} - C_{\min}) \leq 10^{13}$, where K is $n+1$ on a square problem and $2\min(n,m)+1$ on a rectangular one. K is the separation the algorithm needs between the optimum and a 1-optimal matching, and the bound keeps the scaled costs clear of the sentinel marking a forbidden pair and the path sums the solver forms inside 64-bit arithmetic. A matrix whose finite entries are each within 10^{-9} of an integer is taken as an integer matrix and enters at $s = 1$, so the optimum returned is the optimum of the instance as supplied; such a matrix is refused, with the limit named, when its range exceeds $1.25 \times 10^{14}/K$, since no other scale is available to it. Costs that are not integers are solved on the rounded instance, where each cost moved by at most $1/2s$, so the matching returned costs at most $\min(n,m)K(C_{\max} - C_{\min})/10^{13}$ more than the optimum of the matrix as supplied: about 10^{-7} of the range at $n = 1000$ and 10^{-5} of it at $n = 10000$ on a square problem. Potentials are divided by s and shifted back, so they belong to the original matrix, and whether the matching is optimal for it rather than only for the rounded instance is a question `verify_assignment()` answers.

Network-flow methods treat (1) as a minimum-cost flow problem on a bipartite network (Ahuja et al., 1993) and apply push-relabel (Goldberg and Tarjan, 1988), cycle cancelling, network simplex, or Orlin's strongly polynomial algorithm (Orlin, 1993). These are the most general formulations, which makes them the right base for variable-ratio designs where a group absorbs several units. On a plain one-to-one problem they are the slowest choice.

Special-purpose routines cover cases where the general machinery is wasted: exhaustive enumeration below nine units, Hopcroft-Karp bipartite matching (Hopcroft and Karp, 1973) when costs are binary or constant, Ramshaw-Tarjan (Ramshaw and Tarjan, 2012) for strongly rectangular problems where padding to a square matrix would dominate the work, and bottleneck assignment, which minimises the largest pair cost instead of the sum.

The binary case is worth stating in full, since a maximum-cardinality routine carries no costs and the problem does. When every admissible edge holds the same value, all perfect matchings cost the same and any maximum-cardinality matching is optimal. When the finite costs are $\{0,1\}$, the routine runs Hopcroft-Karp on the zero-cost edges alone. A perfect matching there totals zero, which no assignment improves on, so it is optimal for the original costs. If the zero-cost subgraph admits no perfect matching, the optimum needs at least one unit edge, and the problem is passed to the weighted solver on the original matrix. The specialised path answers only where its answer is the optimum.

Two extensions return something other than a single optimal assignment. Murty's ranking procedure (Murty, 1968) with Lawler's partition scheme (Lawler, 1972) enumerates the k cheapest assignments, which lets a user see how much of the total cost is forced and how much is arbitrary. Sinkhorn iteration (Cuturi, 2013) solves the entropy-regularised relaxation, returning a doubly stochastic coupling rather than a permutation.

3 A matching workflow

The package ships `hospital_staff`, a synthetic personnel dataset that supports a treated/control workflow without external data. The `nurses_extended` table holds 200 units and `controls_extended` holds 300, with age, experience, hourly rate, department, certification level and a full-time indicator.

```
library(couplr)
data(hospital_staff)

treated <- transform(hospital_staff$nurses_extended,   id = nurse_id)
control <- transform(hospital_staff$controls_extended, id = nurse_id)
```

`match_couples()` is the front door. It takes the two data frames, the covariates to match on, and returns the optimal one-to-one pairing under the requested distance. Setting `auto_scale = TRUE` runs the preprocessing step: covariates are screened for degenerate variance and excessive missingness, and a scaling method is chosen so that variables measured in years and variables measured in ordinal levels contribute comparably.

```
covars <- c("age", "experience_years", "certification_level")

m <- match_couples(
  left  = treated,
```

```
  right = control,
  vars  = covars,
  auto_scale = TRUE
)
m
```

```
#> Matching Result
#> ===============
#>
#> Method: lap
#> Pairs matched: 200
#> Unmatched (left): 0
#> Unmatched (right): 100
#> Total distance: 59.9496
#> Status: optimal
#>
#> Matched pairs:
#> # A tibble: 200 x 6
#>    left_id right_id distance .age_diff .experience_years_diff
#>    <chr>   <chr>       <dbl>     <dbl>                  <dbl>
#>  1 1       260         0.217        -1                      1
#>  2 2       294         0.175         2                      0
#>  3 3       212         0             0                      0
#>  4 4       444         0.591        -5                      2
#>  5 5       304         0.481        -5                     -1
#>  6 6       273         0             0                      0
#>  7 7       496         0.199         0                      1
#>  8 8       292         0.262         3                      0
#>  9 9       242         0.622        -2                      3
#> 10 10      447         0             0                      0
#> # i 190 more rows
#> # i 1 more variable: .certification_level_diff <int>
```

The result is an S3 object of class `matching_result` with three components. `m$pairs` is a tibble of matched pairs carrying the identifiers, the distance for each pair and a signed per-variable difference column for every matching variable, which makes it possible to see which covariate drives a poor pair without recomputing anything. `m$unmatched` holds the identifiers left over on each side, and `m$info` records the method used, the number of pairs and the total distance.

Balance is the standard for judging whether a matched sample is usable. `balance_diagnostics()` computes standardised mean differences, variance ratios and Kolmogorov-Smirnov statistics on the matched sample, and `balance_table()` formats them.

```
bal <- balance_diagnostics(m, treated, control, vars = covars)
balance_table(bal)
```

```
#> # A tibble: 3 x 7
#>   Variable `Mean Left` `Mean Right` `Mean Diff` `Std Diff` `Var Ratio` `KS Stat`
#>   <chr>          <dbl>        <dbl>       <dbl>      <dbl>       <dbl>     <dbl>
#> 1 age            38.6         39.8       -1.22      -0.116       1.04      0.075
#> 2 experie~        8.46         8.10       0.355      0.074       1.05      0.06
#> 3 certifi~        1.88         1.9       -0.015     -0.021       0.964     0.015
```

Experience and certification fall below the conventional 0.1 threshold on the absolute standardised difference (Stuart, 2010), while age remains slightly above it at 0.116; all three are below 0.15.

`join_matched()` then returns the analysis-ready frame, one row per pair, with every column of both input tables carried through under `_left` and `_right` suffixes, so an outcome model can be fitted directly on the matched data.

```
matched <- join_matched(m, treated, control)
dim(matched)
```

```
#> [1] 200  21
```

From matched pairs to an estimate

The matched frame is the input to an outcome analysis rather than the end of one. We take the hourly rate as the outcome. The paired difference within each matched pair then estimates the association between full-time status and pay among nurses comparable on age, experience and certification.

```
d  <- matched$hourly_rate_left - matched$hourly_rate_right
tt <- t.test(d)
c(estimate = mean(d), lower = tt$conf.int[1], upper = tt$conf.int[2])
```

```
#> estimate    lower    upper
#> 3.531350 1.875651 5.187049
```

Full-time nurses earn 3.53 more per hour than their matched part-time counterparts, with a 95% interval that excludes zero. This is a paired difference on a matched sample and rests on assumptions no matching method can verify: conditional ignorability, positivity, and the stable unit treatment value assumption (SUTVA), under which a unit's outcome depends on its own treatment alone. The matched output can also be passed to an outcome model for regression adjustment; valid estimation and uncertainty quantification still depend on the estimator used (Stuart, 2010).

What matching can offer against the first of those assumptions is a statement about how strong an unmeasured confounder would have to be to overturn the result. `sensitivity_analysis()` implements Rosenbaum bounds for the Wilcoxon signed-rank statistic on one-to-one matched pairs, reporting upper and lower p-value bounds across a grid of the odds-ratio parameter Γ (Rosenbaum, 2002).

```
sens <- sensitivity_analysis(m, treated, control, outcome_var = "hourly_rate")
sens$results[sens$results$gamma %in% c(1, 1.5, 1.75, 2), ]
```

```
#> # A tibble: 4 x 4
#>   gamma t_stat   p_upper  p_lower
#>   <dbl>  <dbl>     <dbl>    <dbl>
#> 1  1    13304. 0.0000187 1.87e- 5
#> 2  1.5  13304. 0.0435    1.01e-11
#> 3  1.75 13304. 0.207     4.48e-15
#> 4  2    13304. 0.481     1.69e-18
```

At $\Gamma = 1$, meaning no hidden bias, the association is significant. It survives an unmeasured confounder that raises one pair member's odds of being full-time by half, $\Gamma = 1.5$, and stops being significant at $\Gamma = 1.75$. A reader can judge from that number whether a confounder of that strength is plausible in this setting; the analysis quantifies exposure to hidden bias rather than removing it.

4 Constraints, designs and interoperability

Reshaping the cost matrix

Three mechanisms restrict which pairs are admissible, and all three work by writing non-finite entries into the cost matrix before it reaches the solver. A global `max_distance` forbids any pair beyond a distance ceiling. Per-variable `calipers` forbid pairs whose raw difference on a named variable exceeds a threshold, independently of the distance metric in use. Explicit forbidden pairs are passed as non-finite entries by the user. Every solver recognises a non-finite entry as a missing edge, so no returned pair ever violates a constraint.

Tight constraints usually make (1) infeasible. couplr first drops the rows and columns whose every edge is forbidden, which are reported unmatched. The remaining submatrix can still fail Hall's condition, with several rows competing for the same few admissible columns, and no assignment reaching every row. Rather than return whichever pairs a heuristic happens to find, couplr replaces the forbidden entries with a finite sentinel σ and re-solves with the same optimal solver.

Write $[\ell, h]$ for the range of the admissible costs of the pruned submatrix and k for the smaller of its two dimensions. Every feasible assignment of the padded problem matches all k rows, so it splits into s sentinel edges and $k - s$ real ones and costs $s\sigma + R$ with $R \in [(k-s)\ell, (k-s)h]$.

Lemma 1. *If $\sigma > k(|\ell| + |h|)$, then a padded assignment using more sentinel edges than another costs strictly more.*

Proof. Let M' and M use s' and s sentinel edges with $s' > s$, and write R' and R for their real costs. Then

$$\text{cost}(M') - \text{cost}(M) = (s' - s)\sigma + R' - R \geq \sigma + (k - s')\ell - (k - s)h \geq \sigma - k|\ell| - k|h| > 0. \qquad \square$$

A minimum-cost solve of the padded problem therefore uses as few sentinel edges as any assignment can, which is a maximum-cardinality admissible matching, and among the assignments attaining that number it minimises R. Dropping the sentinel pairs leaves the lexicographic optimum: the largest admissible matching, cheapest among those of its size. couplr takes $\sigma = (k+1)(|\ell| + |h|) + 1$, which satisfies the hypothesis whatever the signs of ℓ and h and when the admissible costs are all zero, and `assignment(cardinality = "maximum")` prices its dummy columns with the same quantity. A maximisation is solved by negation and the sentinel is negated with it, so the lemma applies to the negated instance; the objective is recomputed over the real pairs, so no sentinel enters a reported cost.

The lemma is about real numbers and the solve is in doubles, so the padded objective has to stay inside the range a double orders exactly. The largest total it can reach is $(k+1)(|\ell| + |h|) + k\sigma$, and above 2^{53} the padding is refused rather than returned as an ordering the arithmetic cannot represent: `match_couples()` warns and falls back to a greedy partial matching, saying that it is not optimal, and `assignment(cardinality = "maximum")` errors and asks for rescaled costs or an explicit `unmatched_penalty`.

```
m_cal <- match_couples(
  treated, control, vars = covars,
  auto_scale   = TRUE,
  calipers     = list(age = 3, experience_years = 2),
  max_distance = 1.5
)
c(pairs = nrow(m_cal$pairs), unmatched_left = length(m_cal$unmatched$left))
```

```
#>          pairs unmatched_left
#>            197              3
```

The caliper costs pairs: 197 of the 200 treated units keep a partner. This is the intended behaviour of a caliper and the reason the unmatched identifiers are returned rather than dropped, since the units that fail to match define the population the estimate actually applies to. The constraints here are severe, leaving 2,173 admissible pairs out of 60,000, so the padded solve is what produces this result; a greedy pass over the same admissible edges matches 180 units rather than 197.

Blocking is the other structural constraint. `block_id` restricts matching to run within levels of a factor, and `matchmaker()` constructs blocks when no grouping variable exists, either from an existing variable or by k-means or hierarchical clustering on chosen block variables. The solver is called once per block, and blocked designs report per-block balance so that imbalance inside one stratum is not averaged away across the others.

Matching designs

Beyond one-to-one matching, `match_couples()` takes `ratio` for k-to-one designs and `replace` for matching with replacement. Five further designs have their own entry points, each returning an object in the same family:

- `ps_match()` estimates a propensity score (Rosenbaum and Rubin, 1983) and matches on it, with the caliper expressed in standard deviations of the linear predictor.
- `full_match()` assigns every unit to a matched group of variable size, so no unit is discarded. The optimal path solves a minimum-cost flow problem using Dijkstra's algorithm with Johnson potentials (Johnson, 1977); a greedy two-pass heuristic is available for large problems.
- `cem_match()` implements coarsened exact matching (Iacus et al., 2012), binning covariates and matching exactly on the binned strata.
- `subclass_match()` divides the sample into propensity-score subclasses and returns subclass weights for estimating the average treatment effect on the treated (ATT) or the average treatment effect (ATE).
- `cardinality_match()` targets the cardinality-matching objective (Zubizarreta et al., 2014), the largest matched sample meeting prespecified balance constraints, with total distance ordered after cardinality. Which solver answers it follows from the kind of balance asked for. Fine and refined covariate balance (Rosenbaum et al., 2007; Pimentel et al., 2015), stated through `fine` and `refined`, are representable in the matching network as category nodes, so a call stating balance

that way reaches a single minimum-cost flow solve that returns the largest balanced sample with a dual certificate at polynomial cost. Linear moment constraints, a finite `max_std_diff` or an explicit `moments` argument, are not representable that way; they are dualised and searched by branch and bound under `node_limit` and `time_limit`. Either way the result reports `n_matched` beside `best_possible`, the largest sample the stated constraints admit, with the `gap` between them, whether the answer is `certified`, and what the search `stopped_on`. A pruning loop is available as `engine = "heuristic"` and reports its gap as unknown, which is the honest reading of what a heuristic establishes.

Interoperability

couplr fits into a workflow built around the established packages. `as_matchit()` converts a couplr result into a `matchit` object, so downstream code written against **MatchIt** keeps working, and `match_data()` returns data in the layout that **MatchIt** users expect. `bal.tab()` methods are registered for the couplr result classes, so **cobalt** produces its usual balance output on a couplr match. Weighted matched data flows into **marginaleffects** (Arel-Bundock, 2025) for estimation.

5 One flow model for every design

Underneath the designs of the previous section there is one object. It has nodes, each carrying a signed supply, and arcs, each carrying a cost and a capacity range. As Table 1 sets out, a one-to-one match, a k-to-one match, matching with replacement and a full match differ in the bounds their arcs receive, not in the code that solves them. Formulating full matching as a minimum-cost flow problem is established (Hansen and Klopfer, 2006); what the flow model contributes here is that every design in the package compiles into it, so one solver, one certificate and one warm start serve all of them.

Table 1: How each matching design is stated in the internal flow model. The designs differ in the capacity bounds their arcs receive and in how many networks are built, rather than in which solver runs.

Design	Flow formulation
One-to-one matching	Unit-capacity bipartite flow
k-to-one matching	Capacitated bipartite flow, treated supply k
Matching with replacement	Control capacities above one
Blocked and exact matching	Independent networks per stratum
Full matching	Lower and upper capacitated circulation
Fine and refined balance	Category nodes with capacity bounds

Compiling a design, solving it, and checking the answer stay three separate steps, because the third is worth something only while it is separate: a solver that hands back its own verdict has proven nothing. The flow and the potentials go into `verify_flow()` as inputs to a check, and the arcs the flow is checked against are stated independently of it.

```
prob <- list(
  n_nodes = 4,
  supply  = c(2, 1, -2, -1),
  arcs = data.frame(tail  = c(1, 1, 2, 2), head  = c(3, 4, 3, 4),
                    lower = c(0, 0, 0, 0), upper = c(2, 2, 2, 2),
                    cost  = c(1, 3, 2, 1))
)
cert <- verify_flow(c(2, 0, 0, 1), prob, potential = c(0, 0, 1, 1))
c(certified = cert$certified_optimal, gap = cert$duality_gap)
```

```
#> certified       gap
#>         1         0
```

Two solvers read that network. One is successive shortest paths with Johnson potentials (Johnson, 1977), reachable as `method = "csflow"`. The other, reachable as `method = "push_relabel"`, is cost-scaling push-relabel in the successive approximation of Goldberg and Tarjan (1990): a sequence of ε-optimal flows, each phase dividing ε, saturating the arcs the smaller ε no longer admits, and clearing the resulting excess with two local operations. A push moves excess along an arc of negative reduced cost; a relabel lowers the price of a node that holds excess and has no such arc, by exactly the amount that gives it one. Below $\varepsilon = 1/(n+1)$ an ε-optimal flow on integer costs is optimal, which is what ends the scaling, so real costs are scaled to integers first, without which the bound says nothing.

Tolerance is where a flow certificate can overstate what it knows, and the scale of the potentials sets it. A design that ranks objectives lexicographically by weighting them, as `cardinality_match()` ranks cardinality above balance above distance, reaches potentials in the millions, where one unit in the last place is around 10^{-9} and an exactly optimal flow cannot meet an absolute tolerance of that size. The comparisons therefore scale with the arc's own magnitude, and the widest tolerance any comparison used comes back as `dual_tolerance`, so the verdict names the resolution it was reached at.

Node potentials therefore come back from every design rather than from the one-to-one case alone. Those potentials are what the next section prices against.

6 Matching without building the graph

A matching problem on n treated units and m controls has nm admissible pairs, and materialising them is what sets the size ceiling: at 50,000 against 100,000 the dense cost matrix alone is 40 GB. The lazy representation of the previous designs removes the matrix while leaving the search over all m columns intact. What `memory_mode = "implicit"` removes is the graph itself.

The loop is column generation (Lübbecke and Desrosiers, 2005) over the pair variables of (1). A restricted master problem holds a subset of them, its optimal duals price the rest at $C_{ij} - u_i - v_j$, and the restricted solution solves the full problem as soon as no omitted variable prices negative. One thing separates it from the textbook setting: the omitted variables are enumerated rather than returned by an optimisation oracle, because a pair variable is an index rather than a combinatorial object to be searched for. Write $E_t \subseteq \{1, \ldots, n\} \times \{1, \ldots, m\}$ for the candidate set at round t and ϵ for the pricing threshold.

1. Seed E_0 with each row's w nearest admissible columns, w read off m as $6\lceil \log_2 m \rceil$ and capped at m.
2. Solve (1) restricted to E_t with the flow model of the previous section, warm-started from the incumbent flow and prices when $t > 0$. If it comes back short of a complete matching, repair the shortfall as below and repeat this step.
3. Read the master's potentials as assignment duals (u, v).
4. Price the omitted pairs: $\bar{C}_{ij} = C_{ij} - u_i - v_j$ for admissible $(i, j) \notin E_t$.
5. Stop if $\bar{C}_{ij} \geq -\epsilon$ everywhere.
6. Otherwise set $E_{t+1} = E_t \cup V_t$, where V_t holds each row's most negative violators, and return to step 2.

The stopping rule is the certificate.

Proposition 1. *Let X solve (1) restricted to a pair set E with optimal duals (u, v), and suppose $C_{ij} - u_i - v_j \geq 0$ for every admissible $(i, j) \notin E$. Then X is optimal for the complete problem.*

Proof. Optimality on E gives complementary slackness for X and (u, v) there, so every matched pair is tight and every unmatched column has $v_j = 0$, and the cost of X is $\sum_i u_i + \sum_j v_j$. The hypothesis extends dual feasibility from E to every admissible pair, so (u, v) is feasible for (2) on the complete problem and that sum is a lower bound on the cost of every assignment of it. X is one of them and attains the bound. □

Nothing in the argument is numerical: the hypothesis is the sign of $\bar{C}_{ij}$ on the omitted pairs, and the conclusion is equality, not proximity. The loop prices against $-\epsilon$ instead of zero, which weakens the conclusion by a stated amount. Under $\bar{C}_{ij} \geq -\epsilon$ the shifted potentials $u_i - \epsilon$ are dual feasible over every admissible pair, and shifting costs the dual objective $n\epsilon$, so the matching's cost is within $n\epsilon$ of the complete optimum. At $\epsilon = 10^{-9}$ and the largest shape in the benchmark below that is 2×10^{-5} in the cost units.

The loop terminates because each pricing round adds at least one pair E_t did not hold: the round continues only when some omitted pair prices below $-\epsilon$, and that pair's row keeps it. So $|E_t|$ strictly increases, and it is bounded by nm. The worst case is that bound: a problem whose optimal duals price most of the grid in makes the loop build the complete graph and pay for the rounds that got there. `max_rounds`, 60 by default, is a guard rather than the termination argument; a run that reaches it returns `iteration_limit` rather than a certificate.

Each complete pricing sweep is $O(nm)$ arithmetic and stores nothing: the source is read one pair at a time and the reduced cost is consumed on the spot, so the memory the mode holds is the candidate set, nw pairs from the seed plus what the rounds add. Rows contribute at most keep_per_row violators each, five by default, so a round adds at most $5n$ pairs and never fewer than one. Selection is by strict improvement on a row's worst kept reduced cost over a scan running columns ascending, so an exact tie keeps the lower column index, and violators enter in (i, j) order: the same input gives the same candidate set at every round.

A master that comes back short of a complete matching has no dual solution to price with, so the shortfall is repaired rather than priced. Hall's condition (Hall, 1935) names a row set S whose neighbourhood $N(S)$ in the restricted graph is too small to go round. Every restricted edge out of S already lands in $N(S)$, so only columns outside it can move the deficiency, and the re-seed reads the full source over the rows of S and takes columns from outside $N(S)$. When there are none, $N(S)$ is the neighbourhood of S in the complete problem as well, and the result is infeasible with $(S, N(S))$ as its witness, which says which units are short of partners rather than reporting only that the problem is infeasible.

So the sparse solution is optimal for a problem that was never built, and certify = TRUE runs that check and returns it. The answer is the assignment a dense solve returns, reached without the memory a dense solve needs.

Two quantities in $search describe what that cost. candidate_edges against possible_edges is the graph that was built against the complete one that was not. edges_evaluated counts distance evaluations over all rounds, so a pair priced in three rounds counts three times and the total can exceed the complete pair count. They move in opposite directions and both belong in the report: the loop holds a small graph by evaluating distances more than once.

Round count is therefore the quantity to control, since each additional round prices every omitted pair again, and it is what the seed rule of step 1 is sized for. A seed too narrow buys the rest of what it needs a round at a time; a seed too wide pays for columns the matching never uses. The rule clears the two-round floor, one seed and one sweep that finds nothing to add and certifies, on the problems in the performance section below, and a run reports the width it used as $search$seed_width.

```
m_imp <- match_couples(
  treated, control, vars = covars,
  auto_scale  = TRUE,
  memory_mode = "implicit",
  certify     = TRUE
)
m_imp$certificate$certified_optimal

#> [1] TRUE

unlist(m_imp$search[c("seed_width", "n_rounds",
                      "candidate_edges", "possible_edges")])

#>      seed_width        n_rounds candidate_edges  possible_edges
#>              54               2           10800           60000
```

The certificate the loop returns is the numerical one, which is why the ϵ version of Proposition 1 is the statement it carries. Dual feasibility over the pairs the master omits comes from the pricing bounds, which ask whether anything prices below $-\epsilon$ rather than evaluating each omitted pair, so the exact conclusion of the certification section is not reachable from a scan assembled that way, and the certificate reports that as its arithmetic.

Constraints tighten the loop rather than complicating it, since a caliper or a distance ceiling is checked before a distance is computed and removes pairs from the set to be priced.

Two neighbours are worth separating from this. Abeywickrama et al. (2021) also decline to compute a complete cost matrix, solving the assignment exactly while deferring exact edge costs behind an inexpensive lower bound; that bound is application-specific, and what it saves is arithmetic. The pricing here needs nothing of the cost function beyond the ability to evaluate a pair, and what it saves is storage, since every omitted pair is still evaluated once per round. The metric-tree pricer in the discussion below is where the two meet. Graded matching (Yu and Rosenbaum, 2022) also enlarges a sparse network only as far as it has to, but a grade is a property of the pairing, fixed before the match; the reduced cost that admits a pair here is a property of the solution the master has just returned, and the loop stops on a statement about the complete graph rather than on the grades running out.

The mode is available on match_couples() and assignment(), and it is requested rather than selected: memory_mode = "auto" does not choose it, because the worst case above is a real one. Its scope is the unit-capacity assignment, because a full match's column nodes carry capacities above one and a column dual there is not the assignment dual the pricer reads.

7 Matching frontiers

A caliper is rarely known in advance. The usual practice is to sweep it, look at what the matched sample and its balance do across the sweep, and choose a point, which means solving the same problem many times over. `match_path()` solves the sweep as one sequence.

```
p <- match_path(
  treated, control, vars = covars,
  auto_scale = TRUE,
  vary   = "max_distance",
  values = c(1.0, 1.2, 1.5, 2.0, 3.0)
)
p$path[, c("max_distance", "n_matched", "total_distance", "certified")]
```

```
#> # A tibble: 5 x 4
#>   max_distance n_matched total_distance certified
#>          <dbl>     <int>          <dbl> <lgl>
#> 1          1         200           62.6 TRUE
#> 2          1.2       200           61.5 TRUE
#> 3          1.5       200           59.9 TRUE
#> 4          2         200           59.9 TRUE
#> 5          3         200           59.9 TRUE
```

The sweep shows where the constraint stops binding: the total distance falls until 1.5 and does not move above it, because a cut that wide forbids nothing the unconstrained optimum uses. Reading that off a path is the point of solving the sweep rather than one value.

Each point resumes from the matching the point before it found. Raising the distance cut admits pairs while leaving that matching feasible, so what remains at the new point is to repair reduced-cost violations on the arcs that just became admissible, which is one round of the pricing loop of the previous section. An independent call at that value pays for a solve from cold. One mechanism serves the sweep and the single solve.

A value whose admissible pairs cannot support a complete matching is reported rather than approximated. Such a point comes back with no matching, with the Hall witness of the previous section beside it, so a sweep that starts below the feasibility threshold says where that threshold is.

The direction is a correctness requirement rather than a convention. `values` must ascend, because a descending sweep withdraws pairs the current matching may be standing on, which breaks feasibility and needs a repair phase the loop does not have. A descending sequence is refused, and the message says why.

The return value is a `couplr_path`. Its `$path` component carries a row per point with the matched count, the total distance, the seconds and rounds that point took, and whether it was certified; `$balance` carries a row per point per variable, so the balance profile across the sweep is a data frame rather than something to assemble by hand. `certify = TRUE` is the default here, and a point's certificate is what says its matching is the optimal one at that value rather than the one the warm start happened to reach.

8 The solver layer

The solver layer is reachable on its own, for a user who arrives with a cost matrix in hand. `lap_solve()` and `assignment()` take a matrix and return the optimal assignment; `lap_solve()` also accepts a long data frame of source, target and cost columns, which is the natural shape when costs arrive from a database. The `method` argument names any of the nineteen solvers; `"auto"` is the default. It accepts twenty names, because `"ssp"` is a second spelling of `"sap"` and resolves to it.

```
set.seed(1)
cost <- matrix(sample.int(100, 25, replace = TRUE), nrow = 5)
res  <- lap_solve(cost)
res
```

```
#> Assignment Result
#> =================
#>
```

```
#> # A tibble: 5 x 3
#>   source target  cost
#>    <int>  <int> <int>
#> 1      1      4     7
#> 2      2      2    14
#> 3      3      1     1
#> 4      4      3    54
#> 5      5      5    44
#>
#> Total cost: 120
#> Method: bruteforce
```

Dual potentials come from `assignment_duals()`, which returns the vectors u and v of (2) alongside the assignment. Passing that result to `verify_assignment()` certifies the matching without a second solve, since the potentials it needs are already in hand:

```
d <- assignment_duals(cost)
verify_assignment(d, cost)$certified_optimal
```

```
#> [1] TRUE
```

Four extensions operate on the same matrix. `lap_solve_kbest()` returns the k cheapest assignments in increasing order of cost, which shows how much of a solution is forced by the data. `bottleneck_assignment()` minimises the largest individual pair cost, the right objective when the worst pair rather than the average pair is what matters. `lap_solve_batch()` solves a stack of independent problems, optionally across threads. `sinkhorn()` returns the entropy-regularised coupling, and `sinkhorn_to_assignment()` rounds it to a permutation.

```
kb <- lap_solve_kbest(cost, k = 3)
tapply(kb$total_cost, kb$rank, unique)
```

```
#>   1   2   3
#> 120 120 150
```

The two cheapest assignments both cost 120 and the third costs 150, so the optimum here is attained by two distinct pairings. A matching study reading the same output would learn that its matched sample is not unique, which stays invisible when a solver returns one assignment.

9 Software design

Layers and object model

The package is organised in three layers. The first converts user input into a cost source: it validates the data frames, screens covariates, applies scaling and weights, and settles how distances are to be obtained and which pairs are admissible. The second compiles the requested design into the flow model and solves it, which for the plain one-to-one case is (1). The third builds result objects and the methods that consume them.

The object model is S3 throughout. Matching results carry the class vector `c("matching_result", "couplr_result")`, with the design-specific classes (`full_matching_result`, `cem_result`, `subclass_result`) sharing the `couplr_result` parent. The shared parent is what makes the diagnostic and conversion layer generic: `balance_diagnostics()`, `join_matched()`, `match_data()` and the `bal.tab()` methods dispatch on it once rather than being rewritten per design. We chose S3 deliberately. S4 or R6 would have bought encapsulation the package does not need, at the cost of making result objects harder to inspect interactively and harder to serialise. Results are plain lists of tibbles, so a user who wants something the package does not provide can reach in and take it.

Solver results are a separate family. `lap_solve()` returns a tibble subclassed as `lap_solve_result` with `source`, `target` and `cost` columns, which means the optimum is immediately usable in **ggplot2** (Wickham, 2016) or a join without an extraction step, while `get_total_cost()` and `get_method_used()` recover the metadata carried in attributes.

C++ organisation

All nineteen solvers are written from scratch in C++ and exposed through **Rcpp** (Eddelbuettel and François, 2011). The `src` tree separates concerns: `src/core` holds the cost-source types and shared utilities with no Rcpp dependency, `src/solvers` holds one algorithm per file with a thin `_rcpp.cpp` wrapper each, `src/flow` holds the flow model, its two solvers and the edge-generation loop, and `src/interface` holds the conversion between R objects and the internal types. The separation means the solver bodies are ordinary C++ that can be compiled and tested outside R, which is what the C++ test suite does.

We template solvers on their cost source rather than writing them against a concrete matrix. A cost source has to provide `at(i, j)`, `allowed(i, j)` and `empty()`. Two types satisfy this interface: `CostMatrix`, which owns a dense flat array plus a mask, and `LazyCostMatrix`, which owns the two feature matrices and computes C_{ij} on demand from the requested metric, applying calipers and the distance ceiling in `allowed()`. A handful of hot loops index the dense mask array directly for speed, which the lazy type cannot support; these are guarded by a `supports_raw_mask` trait and an `if constexpr` branch rather than by duplicating the solver. One solver body therefore serves both cost representations, and there is no second implementation to keep in step. The pricing oracle of the edge-generation loop needs `at(i, j)` and `allowed(i, j)` and nothing else, so it consumes the same concept and adds no third representation of the costs.

`LazyCostMatrix` also bakes in the transformations that the dense path applies in a separate preparation pass, namely mapping forbidden entries to a large sentinel and negating costs when maximising. Doing this at construction is what allows the untouched solver body to work on either type.

Automatic solver selection

`method = "auto"` inspects the problem and picks a solver. The rules are few, and the benchmark in the next section supports rule 5 directly. Rules 1 to 4 are initial heuristics: they follow from the structure each special-purpose routine exploits, and the thresholds in them are not calibrated against a benchmark that varies sparsity or rectangularity.

1. Problems with at most eight rows and eight columns go to exhaustive enumeration, which is exact and faster than setting up a general solver.
2. Cost matrices whose finite entries are constant or lie in $\{0, 1\}$ go to the Hopcroft-Karp style routine, which exploits the absence of a real cost scale.
3. Matrices with more than half of their entries non-finite go to `lapmod`, the sparse Jonker-Volgenant variant, which carries forbidden edges in its adjacency structure at every size.
4. Strongly rectangular problems, with at least three times as many columns as rows, go to shortest augmenting paths, avoiding the padding a square-oriented solver would need.
5. Everything else goes to Jonker-Volgenant.

Rectangular problems are transposed internally so that the solver always sees at least as many columns as rows, and the assignment is mapped back afterwards; the user never sees the transpose.

Rules 2 and 3 read the entries, and so does the rejection of `NaN` inputs that runs for every method, so choosing a solver means a pass over the cost matrix. Expressing that pass in R cost an allocation the size of the matrix for each test, through `any(is.nan())`, `range(finite = TRUE)` and `mean(is.na() | is.infinite())`, and the overhead made `"auto"` up to twice as slow as naming the solver it would pick. A single C++ pass now returns every quantity the rules need, with no allocation and no second scan: the binary-cost test is a branch inside the same loop rather than a `unique()` over the finite entries, and an integer cost matrix is read as integers. That leaves the inspection a linear read in front of a superlinear solve, so its share of the runtime falls as problems grow, and the dashed line in Figure 1 sits on the solver the dispatcher selects. The pass is skippable for callers who already know the shape of their problem, by naming the method.

Memory modes

The dense representation of the cost matrix is the binding constraint on problem size long before runtime is. A 100,000 by 100,000 problem needs 80 GB as doubles; the same units described by 100 covariates need 80 MB. couplr exposes this as `memory_mode`, with values `"dense"`, `"lazy"`, `"implicit"` and `"auto"`.

Under `"lazy"`, no cost matrix is built. The solver holds the feature matrices and evaluates distances as it requests them, which trades arithmetic for memory and returns the same assignment the dense path returns. The lazy path covers Jonker-Volgenant and auction with the built-in metrics, together with calipers and the distance ceiling.

Under "implicit", the edge-generation loop described above runs, and the three modes then differ in what the solver ranges over. The dense path holds the complete matrix. The lazy path holds no matrix and still considers every pair, computing each distance when it is asked for. The implicit path holds a sparse graph and grows it until its duals certify it, so most pairs are priced without ever becoming arcs. All three return the same assignment.

Under "auto", the package decides. Below ten million cells, about 80 MB dense, it skips the check entirely, since probing free memory on every ordinary problem would be pure overhead. Above that, we estimate the dense footprint with a factor of four over the raw eight bytes per cell, which covers the copies made during conversion and preparation that can coexist under garbage-collection lag, and compare that against available system memory. Available memory is read per platform, from `MemAvailable` on Linux, from `vm_stat` page counts on macOS, including inactive and speculative pages because the kernel keeps almost nothing on the free list, and from `Win32_OperatingSystem` on Windows. If the estimate exceeds half of what is available, "auto" switches to the lazy path where one exists and warns explicitly where one does not, naming blocking, greedy matching or a smaller problem as the alternatives. Detection failure is treated as unknown rather than as success: a fixed 4 GB threshold takes over, so the guard never silently disappears on an unrecognised platform.

Verification

Statistical smoke tests establish that a function returns an object of the right shape. For a solver, the corresponding standard is whether the returned assignment is the optimal one. We check every optimal solver against exhaustive enumeration on randomised instances covering integer and fractional costs, square and rectangular shapes, minimisation and maximisation, and matrices containing forbidden edges. The brute-force reference is the same `bruteforce` solver the dispatcher uses below nine units, so the verified path and the shipped path are one implementation.

Constrained matching is checked against the same standard, and needs its own reference because the objective is lexicographic. An enumeration over all partial matchings of a small instance gives the largest admissible matching and the cheapest total among matchings of that size, and the constrained solve has to reproduce both.

The paths that avoid materialising the problem are checked differentially, against the path that materialises it. The implicit loop and the dense solve are run on the same instance and required to return the same total and the same pairing wherever the dense solve can be run at all, and a warm-started point on a path is required to return what an independent solve at that value returns. Two implementations that should agree exactly are the useful kind of oracle, since a disagreement is a defect rather than a matter of degree, and the benchmark in the next section reports both comparisons on the sizes it covers.

Certificates give a third check, and it is the one that survives leaving the test suite. `verify_assignment()` and `verify_flow()` run in $O(nm)$ and $O(\text{arcs})$ respectively without solving anything, so a user can run them on their own problem, in exact arithmetic or at their own tolerance, against a solution the package produced.

The package carries 138 R test files and 37 C++ test files, covering the R layer, the solver bodies compiled outside R, the C++ wrappers, dispatch rules and the matching designs.

Dependency footprint

couplr declares **Rcpp**, **tibble** (Müller and Wickham, 2025), **dplyr**, **rlang**, **purrr** and `methods` in `Imports`, and `Rcpp` alone in `LinkingTo`. The recursive closure of hard dependencies is 17 non-base packages, so a default installation brings 18 packages including couplr itself. The comparable figures on the same day were 18 for **optmatch**, 8 for **MatchIt** and 7 for **designmatch**.

Two choices sit behind that number. First, we use no external solver: all nineteen algorithms are package source, so there is no dependency on an LP or MIP library and no system solver to install. That choice also keeps `LinkingTo` to `Rcpp` alone, which matters because every entry there forces a compile of that package before couplr can be built from source; couplr declares only what its headers actually include. Second, everything that serves a single feature lives in `Suggests` and is loaded at call time. The animation and image-morphing functions, the plotting helpers, the interoperability layer for **MatchIt**, **optmatch** and **cobalt**, and the parallel back end are all optional in this sense, so a user who only wants to match pays for none of them.

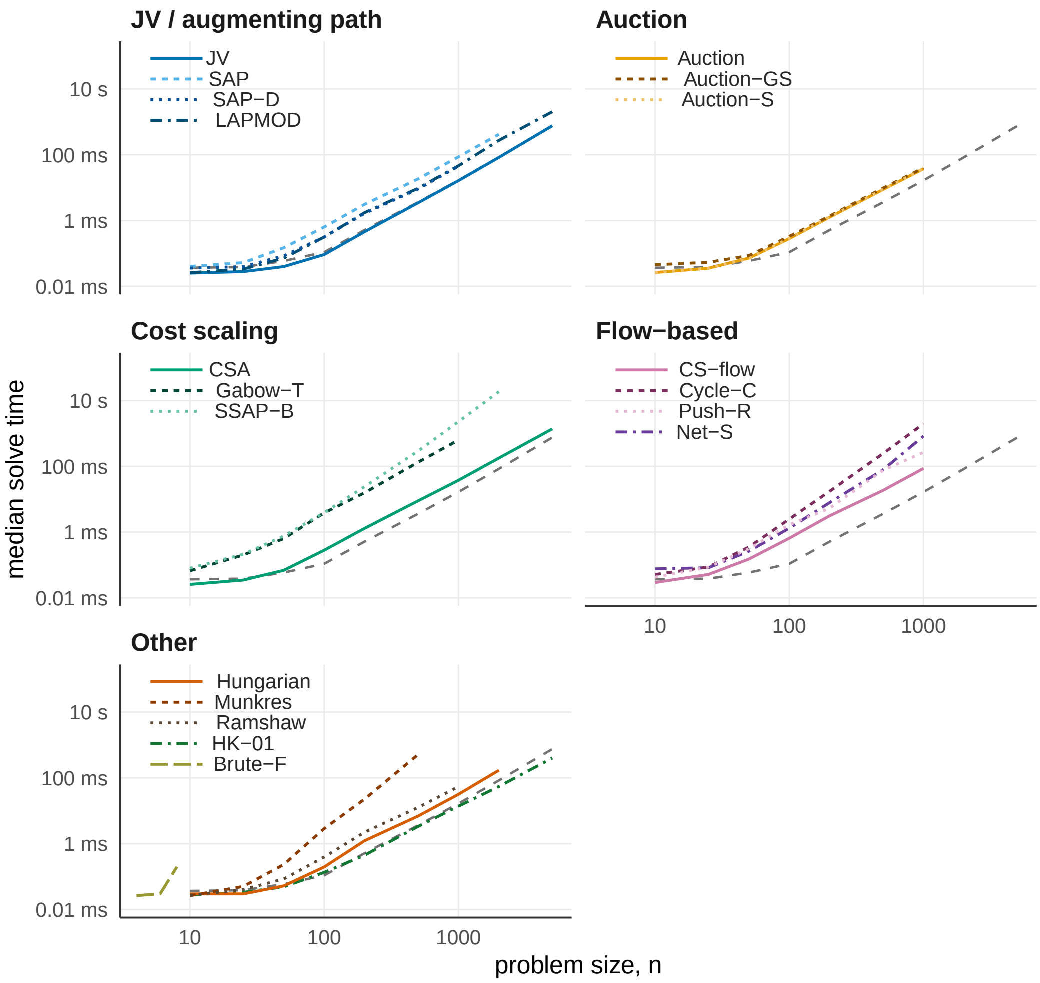

Figure 1: Median wall-clock solve time against problem size for the nineteen assignment solvers in couplr, grouped by algorithm family on shared log-log axes. Seventeen solvers are timed on square integer cost matrices with entries drawn uniformly from 1 to 10,000. The two special-purpose solvers in the Other panel run on their own inputs and are not comparable to the rest: HK-01 is timed on binary cost matrices, and Brute-F only up to n = 8. The dashed grey line repeated in every panel is the automatic dispatcher on the uniform integer costs. Medians of five replicates on a single core of an Apple M4 Pro. Cubic-time and general flow solvers are capped at smaller sizes, which is why their lines end early.

10 Performance

Solver benchmark

Figure 1 reports median wall-clock solve time against problem size for all nineteen solvers, grouped by family. We draw costs as integers uniformly from $[1, 10{,}000]$ on square matrices and report medians of five replicates on a single core of an Apple M4 Pro. The slower families are capped at smaller sizes so that the scalable solvers can be measured to $n = 5000$ within a reasonable budget. All timings in this section were measured on 2026-08-09 under R 4.6.0 with couplr 1.6.2, **optmatch** 0.10.8 and **MatchIt** 4.7.2.

Jonker-Volgenant with its warm start is the fastest general-purpose solver at every measured size, which is why it is the dispatcher's default. The auction and cost-scaling families are competitive at small sizes and fall behind on dense uniform costs, where their advantage, tolerance of degenerate structure and a better asymptotic bound in the Gabow-Tarjan case, does not pay. General flow formulations are the slowest, as expected for machinery that solves a larger problem than the one posed. The two special-purpose routines run on the inputs they exist for, exhaustive enumeration below nine units and the Hopcroft-Karp style solver on binary costs, so their timings are not comparable with the rest of the figure and are shown only to establish that each is practical on its own input.

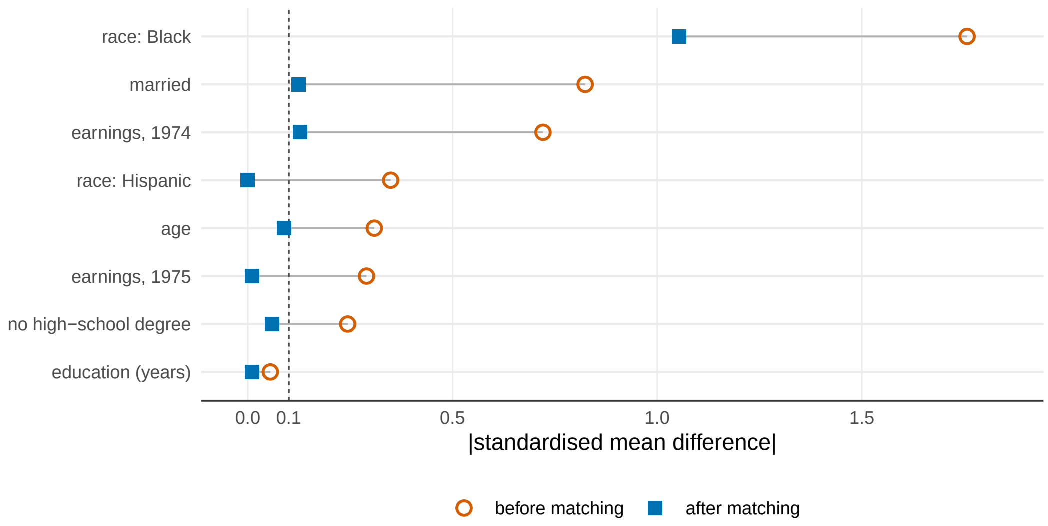

Figure 2: Absolute standardised mean differences on the eight LaLonde NSW covariates, before and after one-to-one optimal Mahalanobis matching with pooled within-group covariance. couplr, MatchIt and optmatch agree to three decimal places after matching, so a single matched point is shown per covariate. The dashed line marks the conventional 0.1 threshold. The indicator for Black respondents starts far outside the plotted range at 1.757 and remains at 1.053 after matching, a residual imbalance that no one-to-one matcher can resolve on this data.

These orderings hold for dense uniform integer costs on square matrices, and that is the whole of what the figure establishes. They fix the dispatcher's default. The remaining rules rest on the structure the special-purpose routines exploit rather than on measurement, and the regimes that would test them, sparse, strongly rectangular and degenerate cost scales, are left to the benchmark extension described in the summary.

Balance against established alternatives

Speed only matters once the packages agree on the answer. We fix the task in Figure 2, one-to-one optimal matching on a Mahalanobis distance with pooled within-group covariance, and run it on the LaLonde NSW data (LaLonde, 1986; Dehejia and Wahba, 1999), 185 treated units, 429 controls and eight covariates. The pooled within-group convention is the one `optmatch::match_on()` uses by default and the one couplr adopts.

The three packages produce identical absolute standardised mean differences to three decimal places, so the matched points coincide and one is plotted. Matching marginals could still hide different assignments, so we score all three on one objective: the same Mahalanobis distance matrix, summed over each package's chosen pairs. Each matches all 185 treated units, at a total distance of 304.042 for couplr against 304.043 for both alternatives. They agree on the optimum, not only on its marginal summaries.

Five of the eight covariates fall below 0.1 after matching; `married` and 1974 earnings sit just above at 0.124 and 0.128, and the indicator for Black respondents drops from 1.757 to 1.053 without approaching the threshold, which is a property of this dataset rather than of any package: the control pool does not contain enough Black respondents for a one-to-one match to balance that indicator.

Scaling

Table 2 reports median wall-clock time on synthetic problems with the same eight-covariate structure, at six sizes with a one-to-two ratio of treated to control units. Every package in this comparison materialises the distance matrix here, so we time couplr with `memory_mode = "dense"`.

The margin widens with problem size. couplr is roughly eight times faster than **optmatch** at $n = 500$ and twenty-seven times faster at $n = 20{,}000$, where it finishes in 10.5 seconds against 4.7 minutes. **MatchIt** runs between ten and twenty-five times slower up to $n = 10{,}000$, which is the largest size it completes before `matchit(method = "optimal")` aborts inside the **optmatch** back end with an integer-size overflow. At $n = 50{,}000$ couplr finishes in 83 seconds and both alternatives exceed the

Table 2: One-to-one optimal Mahalanobis matching, median wall-clock time by problem size. Treated to control ratio 1:2, eight covariates, pooled within-group covariance, single core with single-threaded BLAS on an Apple M4 Pro. Medians of 5, 5, 3, 3, 1 and 1 replicates for the six rows. The optmatch option max.problem.size was set to Inf for the two largest sizes. int overflow marks an integer-size overflow inside the optmatch back end reached through MatchIt; timeout marks a replicate exceeding the 300-second cap.

Problem size	couplr	optmatch	MatchIt
167 + 333	12 ms	97 ms	118 ms
667 + 1,333	141 ms	1.7 s	2.16 s
1,667 + 3,333	720 ms	12.8 s	15.8 s
3,333 + 6,667	2.91 s	59.4 s	72.5 s
6,667 + 13,333	10.5 s	280 s	int overflow
16,667 + 33,333	83.3 s	timeout	timeout

300-second per-replicate cap.

A likely contributor is the difference in formulation rather than a faster inner loop in any single algorithm. optmatch solves a minimum-cost flow problem, which is the right formulation for the full and variable-ratio designs it is built around and more machinery than a plain one-to-one problem needs; couplr's dispatcher sends this dense square problem to Jonker-Volgenant. Separating that contribution from implementation differences would need a benchmark that holds the formulation fixed across packages, which these timings do not do.

The lazy path removes the memory ceiling as well. The same $n = 50{,}000$ match under `memory_mode = "lazy"` completes in 62 seconds without ever building the distance matrix, returning the assignment the dense path returns. At $n = 20{,}000$ it takes 6.4 seconds against 10.5 for the dense path, so on these problems recomputing distances is cheaper than materialising and traversing the matrix.

Edge generation

Table 3 reports the three memory modes on those same problems. The dense and lazy arms are pinned to Jonker-Volgenant, so they differ only in representation; the implicit arm carries the flow model as its restricted master by construction, which is a difference between the arms and is the definition of the mode rather than a choice.

Three things in that table go together. The implicit loop holds a small graph: 16.22% of the pairs at the smallest size and 0.29% at the largest, a share that falls as the problem grows, since the pairs a matching can use grow more slowly than the pairs a problem has. It pays for that in arithmetic, at 1.42 to 2.00 times the complete pair count in distance evaluations, so at eight covariates the loop computes more distances than one full sweep would. And it is still the fastest arm at five of the six sizes, because what it avoids is not the arithmetic but the graph. At 16,667 + 33,333 it finishes in 20 seconds against 62 for the lazy path. At 667 + 1,333 the lazy path is faster, at 55 ms against 64 ms, which is where the

Table 3: One-to-one optimal Mahalanobis matching by memory mode, wall-clock seconds on a single core of an Apple M4 Pro. Graph built is the arcs the implicit loop ended up holding as a share of the arcs the complete problem has. Distances is the number of distance evaluations the loop made over all rounds, as a multiple of the complete pair count, so a pair priced in two rounds counts twice. All three arms were timed in one session, which is a different session from the one Table 2 reports. The dense arm was run at the four sizes where its pairing can be compared against the loop's, which is what this benchmark exists to check; Table 2 carries dense timings at the two largest sizes. Every cell returned the same total distance.

Problem size	dense	lazy	implicit	Graph built	Distances	Rounds
167 + 333	28 ms	97 ms	13 ms	16.22%	2.00	2
667 + 1,333	120 ms	55 ms	64 ms	4.95%	2.00	2
1,667 + 3,333	814 ms	341 ms	287 ms	2.16%	1.96	2
3,333 + 6,667	2.89 s	1.57 s	1.08 s	1.17%	1.88	2
6,667 + 13,333	not run	6.31 s	3.84 s	0.63%	1.71	2
16,667 + 33,333	not run	62.1 s	20.1 s	0.29%	1.42	2

Table 4: A caliper sweep solved as one warm-started path against the same values solved independently, summed per-point solve time on a single core of an Apple M4 Pro. Rounds is the pricing rounds the path took against the rounds the independent solves took, summed over the sweep. The independent solve is a one-point path, so both sides are the same solver reached the same way and differ only in whether a point starts from the point before it.

Problem size	As a path	Independently	Ratio	Rounds
167 + 333	0.027 s	0.073 s	2.67x	22 / 40
667 + 1,333	0.457 s	0.947 s	2.07x	23 / 40
1,667 + 3,333	2.71 s	5.84 s	2.15x	22 / 40
6,667 + 13,333	40.5 s	84.1 s	2.08x	21 / 40

loop's fixed costs are still visible against a problem this size.

Every run in the table settled in 2 rounds, one seed and one sweep that found nothing to add and certified. Every one came back certified with a duality gap of zero. On the four sizes where the dense solve can also be run, the two agree exactly: the totals differ by 0 and the pairing is identical unit by unit, so the loop returns the assignment rather than an assignment of the same cost.

Warm-started paths

Table 4 reports a sweep of 20 caliper values solved as one path against the same 20 values solved independently. The grid is the data's own: its tight end is the tightest caliper the problem still admits a complete matching under, found by bisection, and its wide end is the median treated-to-control distance, which admits half the pairs and constrains nothing.

The saving comes from the rounds column. A warm-started point starts from a matching that is still feasible at the new value, so what it has to repair are the reduced-cost violations on the arcs the wider cut just admitted, which the loop reaches in about one round per point. A point solved independently starts from a seed and pays for the seeding sweep as well.

Correctness is the part of this comparison that has to hold before the timing means anything. Every point on the path returns the status and the matched count its independent solve returns, and the totals agree to 0 relative, so the path is the same sequence of matchings that solving each value from cold would produce.

Feature coverage

Table 5 lists the features that separate the three packages. Rows where all three offer the feature, covariate distance, propensity-score matching, exact blocking and a cost-matrix entry point, are omitted.

11 Summary and discussion

We built couplr on the observation that one optimisation problem sits underneath a set of tasks that R currently treats as unrelated. Making the assignment problem the organising object, rather than a hidden step inside a matching function, is what lets a single package serve a causal-inference user starting from covariates and an operations-research user starting from a cost matrix. Four design consequences follow from that decision. The object model gives the result classes a shared parent, so diagnostics are written once. Every design compiles into one flow model, so what is written for the model serves all of them and node potentials come back from all of them. Those potentials make a solution checkable by a third party, which turns a reported status into a certificate. And a certificate is what makes it safe to solve a problem that was never built, since the loop that grows a sparse graph needs a statement about the complete one to know when to stop.

couplr has five limitations:

- The edge-generation loop's scope is the unit-capacity assignment: a full match's column nodes carry capacities above one, so its column duals are not the assignment duals the pricer reads, and `full_match()` has no implicit path. Its pricing is a block scan over the complete pair set, which keeps memory near-linear while leaving the arithmetic at one pass over the pairs per round, so at eight covariates the loop evaluates more distances than a single full sweep and wins on what it does not build rather than on what it does not compute.

Table 5: Selected assignment-layer features. The table covers the layer this article is about and deliberately omits areas where the alternatives lead, among them the breadth of matching designs reachable through a single MatchIt call and the maturity of optmatch's full-matching implementation. Row meanings: per-variable calipers are marked partial for optmatch because its calipers threshold one distance object at a time, so a per-variable set requires combining objects; rectangular problems are accepted by both alternatives as unequal group sizes, but reach the solver as a padded or variable-ratio formulation rather than as a rectangular cost matrix; the assignment algorithm is user-selectable in optmatch through solver = (RELAX-IV or LEMON, with a choice of LEMON algorithm) and in MatchIt by forwarding solver to optmatch::fullmatch(); the certificate row records whether the package exports a function returning dual variables or verifying optimality, which neither alternative does, and was assessed on 2026-08-26: optmatch attaches the node prices its flow solver ended on to a fullmatch() result in an undocumented MCFSolutions attribute, and the function that evaluates the primal against them is internal. The other rows were assessed 2026-08-09. Both assessments were made against MatchIt 4.7.2 and optmatch 0.10.8.

Feature	couplr	MatchIt	optmatch
Per-variable calipers from a named vector	yes	yes	partial
Rectangular problems without padding	yes	via optmatch	via padding
Sparse cost support	yes	no	yes
k-best assignments	yes	no	no
Bottleneck (minimax) assignment	yes	no	no
Rosenbaum sensitivity bounds	yes	no	no
Dual potentials and optimality certificate	yes	no	no
User-selectable assignment algorithm	19 solvers	via optmatch	2 back ends

- The lazy cost path covers Jonker-Volgenant and auction with the built-in metrics, and user-supplied distance functions still require a dense matrix.
- `cardinality_match()` answers a moment-constrained problem by branch and bound under a node and time limit, so such a run can stop with a gap rather than a certificate, and reports which of the two it reached; the fine and refined balance constraints, which the network represents directly, are answered certified.
- The dispatch rules are calibrated on dense uniform integer costs and on the sparsity and rectangularity boundaries that follow from them; cost structures far from that regime may be better served by naming a solver explicitly, which is why every solver stays reachable by name.
- Matching on observed covariates cannot address unmeasured confounding, and the sensitivity analysis quantifies exposure to it rather than removing it.

Two extensions follow from the limitations above. A metric-tree pricer would attack the arithmetic the block scan leaves in place: pruning a subtree S of controls for row i requires $\min_{j \in S} C_{ij} \geq u_i + \max_{j \in S} v_j$, and Mahalanobis distance is Euclidean after whitening, so one tree serves both metrics. Where such a bound prunes, the pruning is itself the certificate and no complete scan is needed. Calipers are the case for it, since they are axis-aligned boxes in the original covariates and are where matching users actually work. Separately, we are widening the benchmark beyond uniform integer costs, so that the dispatch rules can be recalibrated on sparse, rectangular and clustered cost matrices rather than resting on the structure each routine exploits.

couplr is available on CRAN and developed at `https://github.com/gcol33/couplr`, with documentation at `https://gillescolling.com/couplr/`. Bug reports and feature requests go through the GitHub issue tracker.

12 Acknowledgements

No external funding supported this work.

Bibliography

T. Abeywickrama, V. Liang, and K.-L. Tan. Optimizing bipartite matching in real-world applications by incremental cost computation. *Proceedings of the VLDB Endowment*, 14(7):1150–1158, 2021. doi: 10.14778/3450980.3450983. [p11]

R. K. Ahuja, T. L. Magnanti, and J. B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, Englewood Cliffs, NJ, 1993. [p5]

V. Arel-Bundock. *marginaleffects: Predictions, Comparisons, Slopes, Marginal Means, and Hypothesis Tests*, 2025. URL `https://CRAN.R-project.org/package=marginaleffects`. R package version 0.29.0. [p9]

M. Berkelaar et al. *lpSolve: Interface to Lp_solve v. 5.5 to Solve Linear/Integer Programs*, 2024. URL `https://CRAN.R-project.org/package=lpSolve`. R package version 5.6.23. [p1]

D. P. Bertsekas. The auction algorithm: A distributed relaxation method for the assignment problem. *Annals of Operations Research*, 14:105–123, 1988. doi: 10.1007/BF02186476. [p4]

G. Birkhoff. Tres observaciones sobre el algebra lineal. *Universidad Nacional de Tucumán Revista Series A*, 5:147–151, 1946. [p3]

R. E. Burkard and E. Çela. Linear assignment problems and extensions. *Handbook of Combinatorial Optimization*, pages 75–149, 1999. doi: 10.1007/978-1-4757-3023-4_2. [p3]

G. Colling. *couplr: Optimal Pairing and Matching via Linear Assignment*, 2026. URL `https://CRAN.R-project.org/package=couplr`. R package version 1.5.5. [p2]

M. Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems*, volume 26, pages 2292–2300, 2013. URL `https://papers.nips.cc/paper_files/paper/2013/hash/af21d0c97db2e27e13572cbf59eb343d-Abstract.html`. [p5]

R. H. Dehejia and S. Wahba. Causal effects in nonexperimental studies: Reevaluating the evaluation of training programs. *Journal of the American Statistical Association*, 94(448):1053–1062, 1999. doi: 10.1080/01621459.1999.10473858. [p17]

D. Eddelbuettel and R. François. Rcpp: Seamless R and C++ integration. *Journal of Statistical Software*, 40(8):1–18, 2011. doi: 10.18637/jss.v040.i08. [p14]

H. N. Gabow and R. E. Tarjan. Faster scaling algorithms for network problems. *SIAM Journal on Computing*, 18(5):1013–1036, 1989. doi: 10.1137/0218069. [p4]

A. V. Goldberg and R. Kennedy. An efficient cost scaling algorithm for the assignment problem. *Mathematical Programming*, 71(2):153–177, 1995. doi: 10.1007/BF01585996. [p4]

A. V. Goldberg and R. E. Tarjan. A new approach to the maximum-flow problem. *Journal of the ACM*, 35(4):921–940, 1988. doi: 10.1145/48014.61051. [p5]

A. V. Goldberg and R. E. Tarjan. Finding minimum-cost circulations by successive approximation. *Mathematics of Operations Research*, 15(3):430–466, 1990. doi: 10.1287/moor.15.3.430. [p9]

N. Greifer. *cobalt: Covariate Balance Tables and Plots*, 2025. URL `https://CRAN.R-project.org/package=cobalt`. R package version 4.6.3. [p1]

P. Hall. On representatives of subsets. *Journal of the London Mathematical Society*, s1-10(1):26–30, 1935. doi: 10.1112/jlms/s1-10.37.26. [p11]

B. B. Hansen. Optmatch: Flexible, optimal matching for observational studies. *R News*, 7(2):18–24, 2007. URL `https://journal.r-project.org/articles/RN-2007-014/`. [p1]

B. B. Hansen and S. O. Klopfer. Optimal full matching and related designs via network flows. *Journal of Computational and Graphical Statistics*, 15(3):609–627, 2006. doi: 10.1198/106186006X137047. [p1, 9]

D. E. Ho, K. Imai, G. King, and E. A. Stuart. MatchIt: Nonparametric preprocessing for parametric causal inference. *Journal of Statistical Software*, 42(8):1–28, 2011. doi: 10.18637/jss.v042.i08. [p1]

J. E. Hopcroft and R. M. Karp. An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs. *SIAM Journal on Computing*, 2(4):225–231, 1973. doi: 10.1137/0202019. [p5]

K. Hornik. *clue: Cluster Ensembles*, 2024. URL `https://CRAN.R-project.org/package=clue`. R package version 0.3-66. [p1]

S. M. Iacus, G. King, and G. Porro. Causal inference without balance checking: Coarsened exact matching. *Political Analysis*, 20(1):1–24, 2012. doi: 10.1093/pan/mpr013. [p8]

D. B. Johnson. Efficient algorithms for shortest paths in sparse networks. *Journal of the ACM*, 24(1):1–13, 1977. doi: 10.1145/321992.321993. [p8, 9]

R. Jonker and T. Volgenant. A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, 38(4):325–340, 1987. doi: 10.1007/BF02278710. [p4]

H. W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2 (1-2):83–97, 1955. doi: 10.1002/nav.3800020109. [p4]

R. J. LaLonde. Evaluating the econometric evaluations of training programs with experimental data. *The American Economic Review*, 76(4):604–620, 1986. URL https://www.jstor.org/stable/1806062. [p17]

E. L. Lawler. A procedure for computing the K best solutions to discrete optimization problems and its application to the shortest path problem. *Management Science*, 18(7):401–405, 1972. doi: 10.1287/mnsc.18.7.401. [p5]

M. E. Lübbecke and J. Desrosiers. Selected topics in column generation. *Operations Research*, 53(6): 1007–1023, 2005. doi: 10.1287/opre.1050.0234. [p10]

K. Müller and H. Wickham. *tibble: Simple Data Frames*, 2025. URL https://CRAN.R-project.org/package=tibble. R package version 3.3.0. [p15]

J. Munkres. Algorithms for the assignment and transportation problems. *Journal of the Society for Industrial and Applied Mathematics*, 5(1):32–38, 1957. doi: 10.1137/0105003. [p4]

K. G. Murty. An algorithm for ranking all the assignments in order of increasing cost. *Operations Research*, 16(3):682–687, 1968. doi: 10.1287/opre.16.3.682. [p5]

J. B. Orlin. A faster strongly polynomial minimum cost flow algorithm. *Operations Research*, 41(2): 338–350, 1993. doi: 10.1287/opre.41.2.338. [p5]

S. D. Pimentel. *rcbalance: Large, Sparse Optimal Matching with Refined Covariate Balance*, 2022. URL https://CRAN.R-project.org/package=rcbalance. R package version 1.8.8. [p1]

S. D. Pimentel, R. R. Kelz, J. H. Silber, and P. R. Rosenbaum. Large, sparse optimal matching with refined covariate balance in an observational study of the health outcomes produced by new surgeons. *Journal of the American Statistical Association*, 110(510):515–527, 2015. doi: 10.1080/01621459.2014.997879. [p1, 8]

L. Ramshaw and R. E. Tarjan. On minimum-cost assignments in unbalanced bipartite graphs. Technical Report HPL-2012-40R1, HP Laboratories, 2012. URL https://www.hpl.hp.com/techreports/2012/HPL-2012-40R1.html. [p5]

P. R. Rosenbaum. *Observational Studies*. Springer Series in Statistics. Springer, 2 edition, 2002. doi: 10.1007/978-1-4757-3692-2. [p7]

P. R. Rosenbaum and D. B. Rubin. The central role of the propensity score in observational studies for causal effects. *Biometrika*, 70(1):41–55, 1983. doi: 10.1093/biomet/70.1.41. [p8]

P. R. Rosenbaum, R. N. Ross, and J. H. Silber. Minimum distance matched sampling with fine balance in an observational study of treatment for ovarian cancer. *Journal of the American Statistical Association*, 102(477):75–83, 2007. doi: 10.1198/016214506000001059. [p8]

F. Sävje, J. S. Sekhon, and M. Higgins. *quickmatch: Quick Generalized Full Matching*, 2024. URL https://CRAN.R-project.org/package=quickmatch. R package version 0.2.3. [p1]

J. S. Sekhon. Multivariate and propensity score matching software with automated balance optimization: The Matching package for R. *Journal of Statistical Software*, 42(7):1–52, 2011. doi: 10.18637/jss.v042.i07. [p1]

J. R. Shewchuk. Adaptive precision floating-point arithmetic and fast robust geometric predicates. *Discrete & Computational Geometry*, 18(3):305–363, 1997. doi: 10.1007/PL00009321. [p4]

E. A. Stuart. Matching methods for causal inference: A review and a look forward. *Statistical Science*, 25(1):1–21, 2010. doi: 10.1214/09-STS313. [p6, 7]

H. Wickham. *ggplot2: Elegant Graphics for Data Analysis*, 2016. URL https://ggplot2.tidyverse.org. [p13]

R. Yu and P. R. Rosenbaum. Graded matching for large observational studies. *Journal of Computational and Graphical Statistics*, 31(4):1406–1415, 2022. doi: 10.1080/10618600.2022.2058001. [p2, 11]

R. Yu, J. H. Silber, and P. R. Rosenbaum. Matching methods for observational studies derived from large administrative databases. *Statistical Science*, 35(3):338–355, 2020. doi: 10.1214/19-STS699. [p2]

J. R. Zubizarreta, R. D. Paredes, and P. R. Rosenbaum. Matching for balance, pairing for heterogeneity in an observational study of the effectiveness of for-profit and not-for-profit high schools in Chile. *The Annals of Applied Statistics*, 8(1):204–231, 2014. doi: 10.1214/13-AOAS713. [p8]

J. R. Zubizarreta, C. Kilcioglu, and J. P. Vielma. *designmatch: Matched Samples that are Balanced and Representative by Design*, 2024. URL https://CRAN.R-project.org/package=designmatch. R package version 0.5.4. [p1]

Gilles Colling
University of Vienna
Department of Botany and Biodiversity Research
Rennweg 14, 1030 Vienna, Austria
https://gillescolling.com
ORCiD: 0000-0003-3070-6066
gilles.colling051@gmail.com